

PHASE 1 DOCUMENTATION REWRITE

Windows 98 Portfolio OS

Build Documentation — Explained From Current Code

Document	Build Documentation — Explained From Current Code
System	Windows 98 Portfolio OS
Generated	June 20, 2026 — from Markdown and repository source
Source of truth	Repository source files are authoritative; Markdown docs are background reference where details differ.
Page geometry	US Letter, 1 inch margins, repeating table header rows.

This version is not a stitched Markdown export. It reads the Markdown docs as background, checks the current source files, and explains how the current app is assembled, booted, rendered, styled, persisted, tested, and integrated.

1 What This Project Is

The project is a browser-hosted Windows 98 style portfolio operating system. It is not only a themed landing page: it has a boot sequence, BIOS setup, boot-device menu, desktop shell, taskbar, Start menu, draggable windows, a virtual C: drive, system recovery, persisted settings, and 25 app definitions.

Portfolio content is embedded into the simulated OS instead of being presented as ordinary web sections. About, Projects, Contact, resume content, media files, games, and help content all appear through app windows, desktop icons, Start menu entries, or virtual filesystem nodes.

Current implementation summary: React 19 + TypeScript + Vite; one Context/reducer OS store; pure subsystem modules in src/os; data registries in src/data; eager shell CSS plus lazy per-app CSS; localStorage persistence key win98-portfolio.v4.

Current fact	Verified from source
25 app definitions	src/data/apps.ts
9 OS phases: boot, biosSetup, bootDeviceMenu, bootMenu, desktop, dosOnly, recovery, crashed, shutdown	src/types.ts
17 eager CSS imports before app chunks	src/main.tsx
Persistence key win98-portfolio.v4	src/os/persistence.ts
BIOS setup and boot-device menu are current features	src/types.ts, src/App.tsx, src/data/bios.ts
Self-hosted DOS games run through js-dos	public/js-dos, public/games, JsDosGameApp.tsx

2 Build Shape: Entry Point, Provider, Router

The browser entry point is deliberately small. It imports the third-party 98.css base theme, imports the global shell styles in a stable order, then mounts React under OsProvider. OsProvider is the source of truth for the simulated OS; App is the phase router and shell surface.

`src/main.tsx` *root composition*

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import '98.css/dist/98.css'
import './index.css'
// Eager/global styles, imported here in original-source order so the cascade matches the
// pre-split App.css. App-specific styles are imported lazily inside each app component.
import './styles/base.css'
import './styles/cursors.css'
import './components/system/BootScreen.css'
import './styles/desktop.css'
import './components/shell/WindowFrame.css'
import './styles/common.css'
import './styles/file-manager.css'
import './components/system/CrashScreen.css'
import './components/shell/StartMenu.css'
import './components/shell/Taskbar.css'
import './components/shell/DesktopContextMenu.css'
import './styles/responsive.css'
import './styles/system-screens.css'
import './styles/bios.css'
import './styles/message-box.css'
import App from './App.tsx'
import { OsProvider } from './os/store'
```

The important design choice is that App does not own global OS state. It asks useOs for the current phase and renders the matching surface: boot, BIOS setup, boot device menu, boot menu, desktop, DOS-only terminal, recovery console, crash screen, or shutdown screen.

`src/App.tsx` *phase router*

```
function App() {
  const { state, restart } = useOs()

  switch (state.phase) {
    case 'boot':
      return <BootScreen />
    case 'biosSetup':
      return <BiosSetupScreen />
    case 'bootDeviceMenu':
      return <BootDeviceQuickMenu />
    case 'bootMenu':
      return <BootMenu />
    case 'desktop':
      return <Desktop />
    case 'dosOnly':
      return (
        <main className="dos-only-screen">
          <TerminalApp windowId="dos-only" payload={{ path: 'C:\\' }} />
        </main>
      )
    case 'recovery':
      return <RecoveryConsole />
    case 'crashed':
      return <CrashScreen crash={state.crash ?? {
        title: 'Windows protection error',
        message: 'Windows could not continue.',
        detail: 'A simulated fatal exception occurred.',
        stopCode: '0E : 0028 : C0011E36',
        crashedAt: new Date().toLocaleString(),
      }} onRestart={() => restart('bootMenu')} />
    case 'shutdown':
      return <ShutdownScreen />
    default:
      return <Desktop />
  }
}
```

```
}
}
```

3 The State Contract

The current source centralizes shared language in `src/types.ts`. That file defines app IDs, icon keys, window payloads, BIOS settings, filesystem nodes, network state, audio state, crash state, and the full `OsState` object. This is why separate apps can talk to the shell without inventing their own mini-protocols.

Type area	What it controls in the app
AppId / AppDefinition	The 25 programs the shell can launch, including desktop apps, tools, games, portfolio screens, and Help.
WindowPayload	Context passed into windows: <code>filePath</code> , folder path, URL, <code>projectId</code> , command, control-panel section, or custom <code>windowTitle</code> .
FsState / FsNode	The virtual C: drive, recycle bin, app launchers, <code>dataUrl</code> media files, hidden/passcode folders, and system-file metadata.
OsPhase / BootMode / BiosSettings	The startup model, BIOS setup, boot-device menu, normal/safe/DOS/recovery targets, and persisted BIOS choices.
OsContextValue	The facade API exposed by <code>useOs</code> : open windows, mutate files, manage network/theme/audio, boot/restart/shutdown.

Components reach the OS through one facade hook, `useOs`. That hook only returns the context built by `OsProvider`. If a component is outside the provider, it throws immediately instead of silently failing.

`src/os/useOs.ts` *facade hook contract*

```
WindowRect,
} from '../types'

// Taglish note: separate file ito para stable ang Fast Refresh. Provider lives
// sa store.tsx, pero hook/context contract nandito para hindi mixed exports.
export type OsContextValue = {
  state: OsState
  // windows
  openApp(appId: AppId, payload?: WindowPayload): void
  openNode(path: string): void
  closeWindow(id: string): void
  minimizeWindow(id: string): void
  focusWindow(id: string): void
  toggleMaximize(id: string): void
  moveWindow(id: string, rect: WindowRect): void
  setWindowTitle(id: string, title: string): void
  // desktop / shell
  setStartMenuOpen(open: boolean): void
  moveDesktopIcon(id: string, pos: Point): void
  arrangeDesktopIcons(): void
  showMessageBox(req: Omit<MessageBoxRequest, 'id'>): void
  dismissMessageBox(id: string, button: MessageBoxButton): void
  // filesystem
  fsOps: {
    createFolder(parent: string, name: string): string | null
    createFile(parent: string, name: string, opts?: { content?: string; dataUrl?: string }): void
    writeFile(path: string, data: { content?: string; dataUrl?: string }): string | null
    renameNode(path: string, newName: string): string | null
    moveNode(path: string, targetFolder: string): string | null
    copyNode(path: string, targetFolder: string): string | null
    createDesktopShortcut(targetPath: string): string | null
    deleteNode(path: string, opts?: { skipConfirm?: boolean }): string | null
    restoreEntry(entryId: string): string | null
    emptyRecycleBin(): void
  }
}
```

```
    replaceFs(fs: FsState): void
  }
  setClipboard(c: ClipboardState): void
  // network
  networkOps: {
    connect(): void
    disconnect(): void
    renewDhcp(): void
    applyConfig(partial: Partial<NetworkState>): void
    recordTraffic(sent: number, received: number): void
  }
  // appearance
  setTheme(themeId: string): void
  setWallpaper(wallpaperId: string): void
  setCursorScheme(scheme: CursorSchemeId): void
  // audio
  enableAudio(): void
  setAudioMuted(muted: boolean): void
  setAudioVolume(volume: number): void
  playSound(id: SoundId): void
  // system
  crashSystem(crash: CrashState): void
  enterBiosSetup(): void
  enterBootDeviceMenu(): void
```

4 OsProvider and the Reducer

OsProvider initializes the simulated machine, keeps a reducer-driven OsState, exposes safe helper functions, performs browser effects, and debounces persistence. The reducer owns phase/window/state transitions; components and apps call helper functions rather than editing state directly.

`src/os/store.tsx` *default state creation*

```
function createDefaultState(): OsState {
  const persisted = loadPersistedState()
  const themeId = persisted?.themeId ?? defaultThemeId
  const fs = persisted?.fs ? ensurePortfolioSeedFiles(persisted.fs) : createInitialFsState()
  return {
    phase: 'boot',
    bootMode: 'normal',
    bootProfile: 'cold',
    bootTarget: 'normal',
    bios: persisted?.bios ?? defaultBiosSettings,
    fs,
    windows: [],
    activeWindowId: undefined,
    zCounter: 20,
    network: persisted?.network ?? defaultNetworkState,
    themeId,
    wallpaperId: persisted?.wallpaperId ?? getTheme(themeId).wallpaperId ?? defaultWallpaperId,
    cursorScheme: persisted?.cursorScheme ?? 'win98',
    audio: { enabled: true, muted: false, volume: persisted?.audio.volume ?? 0.7 },
    crash: null,
    desktopIcons:
      persisted?.desktopIcons && Object.keys(persisted.desktopIcons).length
      ? persisted.desktopIcons
      : defaultDesktopIconPositions(),
    clipboard: null,
    messageBoxes: [],
    startMenuOpen: false,
  }
}

// -----
// Reducer
// -----
```



```
// Taglish note: reducer lang ang dapat magpalit ng core OS phase/window state.
```

The provider also keeps `stateRef` synchronized with the latest state. That is why filesystem operations and rapid window opens can read the latest committed state synchronously before dispatching another action. It prevents stale-state bugs during quick bursts of user input.

Window creation is guarded twice. `openApp` checks the current windows through `stateRef`, and the reducer still has an `OPEN_WINDOW` idempotency guard. This matters because two event handlers can fire in the same tick and otherwise create duplicate windows with the same React key.

`src/os/store.tsx` *instance identity and duplicate-window guard*

```
function instanceIdFor(appId: AppId, payload?: WindowPayload): string {
  const singleton = appDefinitions[appId].singleton === false
  if (appId === 'explorer') {
    return `explorer:${normalizePath(payload?.path ?? 'C:\\').toLowerCase()}`
  }
  if (appId === 'projectDetails' && payload?.projectId) {
    return `projectDetails:${payload.projectId}`
  }
  if (appId === 'notepad' && payload?.filePath) {
    return `notepad:${normalizePath(payload.filePath).toLowerCase()}`
  }
  if (appId === 'wordpad' && payload?.filePath) {
    return `wordpad:${normalizePath(payload.filePath).toLowerCase()}`
  }
  if (appId === 'imageView' && payload?.filePath) {
    return `imageView:${normalizePath(payload.filePath).toLowerCase()}`
  }
  if (appId === 'videoPlayer' && payload?.filePath) {
    return `videoPlayer:${normalizePath(payload.filePath).toLowerCase()}`
  }
  if (singleton) {
    return appId
  }
  instanceCounter += 1
  return `${appId}#${instanceCounter}`
}

case 'OPEN_WINDOW': {
  const zIndex = state.zCounter + 1
  // Idempotent guard: if a window with this instanceId already exists, focus
  // it instead of appending a duplicate. openApp dedupes against stateRef, but
  // two synchronous opens (e.g. a key press firing two handlers in one tick)
  // both read the same stale state – without this they'd create two windows
  // sharing one React key.
```

```
const existing = state.windows.find((win) => win.instanceId === action.window.instanceId)
if (existing) {
  return {
    ...state,
    zCounter: zIndex,
    windows: state.windows.map((win) =>
      win.instanceId === action.window.instanceId ? { ...win, minimized: false, zIndex
    },
    activeWindowId: action.window.instanceId,
    startMenuOpen: false,
  }
}
return {
  ...state,
  zCounter: zIndex,
  windows: [...state.windows, { ...action.window, zIndex }],
  activeWindowId: action.window.instanceId,
  startMenuOpen: false,
```

5 Window and App Launch Flow

The shell is data-driven. `appDefinitions` defines each app's title, icon, default rectangle, singleton behavior, and Safe Mode availability. `desktopIconDefs` and `startMenuModel` are launch surfaces that reference those app IDs. Adding a launcher is usually a data edit; adding a new program also requires a lazy import and render case in `App.tsx`.

Launch source	How it reaches a window
Desktop icon	DesktopIcon calls openApp(applId, payload) from the icon definition.
Start menu item	StartMenu renders startMenuModel and calls openApp with the configured payload.
Filesystem node	openNode reads the FsNode, openTargetFor resolves its association, then openApp launches the app.
Terminal command	executeCommand returns an openApp effect; Terminal applies the effect through useOs.
Project card / app UI	Feature apps call openApp directly for drill-down windows such as Project Details.

`src/os/store.tsx` *openApp behavior*

```
(appId: AppId, payload?: WindowPayload) => {
  const current = stateRef.current
  const def = appDefinitions[appId]
  if (current.bootMode === 'safe' && current.phase === 'desktop' && def.safeModeAvailable) {
    showMessageBox({
      title: def.title,
      message: `Windows cannot run ${def.title} in Safe Mode.`,
      detail: 'Restart your computer normally to use this program.',
      icon: 'error',
      buttons: ['ok'],
    })
    return
  }
  const instanceId = instanceIdFor(appId, payload)
  const existing = current.windows.find((win) => win.instanceId === instanceId)
  if (existing) {
    if (payload && JSON.stringify(payload) !== JSON.stringify(existing.payload)) {
      dispatch({
        type: 'UPDATE_WINDOW_PAYLOAD',
        id: instanceId,
        payload,
        title: titleFor(appId, current.fs, payload),
      })
    }
    focusWindow(instanceId)
    return
  }
  const rect = clampRect(def.defaultRect, (current.windows.length % 8) * 22)
  dispatch({
    type: 'OPEN_WINDOW',
    window: {
      instanceId,
      appId,
      title: titleFor(appId, current.fs, payload),
    },
  })
}
```

```
        icon: iconFor(appId, payload),
        minimized: false,
        maximized: false,
        payload,
        ...rect,
      },
    })
    playSound('launch')
  },
  [focusWindow, playSound, showMessageBox],
)
```

App rendering is lazy. The shell imports each app with `React.lazy`, and `renderAppWindow` maps the `appId` to the correct component. That keeps boot, desktop, taskbar, and window chrome eager while loading app JavaScript and app CSS only when the user opens that app.

src/App.tsx *lazy app registration and render switch*

```
const AboutApp = lazy(() => import('./components/apps/AboutApp').then((m) => ({ default: m.A
const CalculatorApp = lazy(() => import('./components/apps/CalculatorApp').then((m) => ({ de
const ContactApp = lazy(() => import('./components/apps/ContactApp').then((m) => ({ default:
const ControlPanelApp = lazy(() =>
  import('./components/apps/ControlPanelApp').then((m) => ({ default: m.ControlPanelApp })),
)
const CreditsApp = lazy(() => import('./components/apps/CreditsApp').then((m) => ({ default:
const ExplorerApp = lazy(() => import('./components/apps/ExplorerApp').then((m) => ({ default:
const GalleryApp = lazy(() => import('./components/apps/GalleryApp').then((m) => ({ default:
const HelpApp = lazy(() => import('./components/apps/HelpApp').then((m) => ({ default: m.Hel
const ImageViewerApp = lazy(() => import('./components/apps/ImageViewerApp').then((m) => ({
const InternetExplorerApp = lazy(() =>
  import('./components/apps/InternetExplorerApp').then((m) => ({ default: m.InternetExplorer
)
const MediaPlayerApp = lazy(() => import('./components/apps/MediaPlayerApp').then((m) => ({
const JsDosGameApp = lazy(() =>
  import('./components/apps/JsDosGameApp').then((m) => ({ default: m.JsDosGameApp })),
)
const MinesweeperApp = lazy(() => import('./components/apps/MinesweeperApp').then((m) => ({
const NetworkApp = lazy(() => import('./components/apps/NetworkApp').then((m) => ({ default:
const NotepadApp = lazy(() => import('./components/apps/NotepadApp').then((m) => ({ default:
const PaintApp = lazy(() => import('./components/apps/PaintApp').then((m) => ({ default: m.P
const ProjectDetailsApp = lazy(() =>
  import('./components/apps/ProjectDetailsApp').then((m) => ({ default: m.ProjectDetailsApp
)
const ProjectsApp = lazy(() => import('./components/apps/ProjectsApp').then((m) => ({ default:
const RecycleBinApp = lazy(() => import('./components/apps/RecycleBinApp').then((m) => ({ de
const RunDialogApp = lazy(() => import('./components/apps/RunDialogApp').then((m) => ({ defa
const SoundRecorderApp = lazy(() =>
  import('./components/apps/SoundRecorderApp').then((m) => ({ default: m.SoundRecorderApp })
)
const TaskManagerApp = lazy(() => import('./components/apps/TaskManagerApp').then((m) => ({
const TerminalApp = lazy(() => import('./components/apps/TerminalApp').then((m) => ({ default:
const VideoPlayerApp = lazy(() => import('./components/apps/VideoPlayerApp').then((m) => ({
```

```
const WordPadApp = lazy(() => import('./components/apps/WordPadApp')).then((m) => ({ default:

function renderAppWindow(win: WindowState, openApp: {appId: AppId, payload?: WindowPayload})
  const props = { windowId: win.instanceId, payload: win.payload }
  switch (win.appId) {
    case 'explorer':
      return <ExplorerApp {...props} />
    case 'recycleBin':
      return <RecycleBinApp />
    case 'terminal':
      return <TerminalApp {...props} />
    case 'notepad':
      return <NotepadApp {...props} />
    case 'wordpad':
      return <WordPadApp {...props} />
    case 'paint':
      return <PaintApp {...props} />
    case 'imageViewer':
      return <ImageViewerApp key={win.payload?.filePath ?? win.instanceId} {...props} />
    case 'internetExplorer':
      return <InternetExplorerApp {...props} />
    case 'mediaPlayer':
      return <MediaPlayerApp key={win.payload?.filePath ?? win.payload?.url ?? win.instanceId} {...props} />
    case 'videoPlayer':
      return <VideoPlayerApp key={win.payload?.filePath ?? win.payload?.url ?? win.instanceId} {...props} />
    case 'gallery':
      return <GalleryApp />
    case 'soundRecorder':
      return <SoundRecorderApp />
    case 'controlPanel':
      return <ControlPanelApp key={win.payload?.controlPanelSection ?? 'controlPanel'} {...props} />
    case 'network':
      return <NetworkApp />
    case 'run':
      return <RunDialogApp />
    case 'taskManager':
```



```
    return <TaskManagerApp />
  case 'calculator':
    return <CalculatorApp />
  case 'minesweeper':
    return <MinesweeperApp />
  case 'dosGame':
    return <JsDosGameApp {...props} />
  case 'about':
    return <AboutApp />
  case 'contact':
    return <ContactApp />
  case 'projects':
    return <ProjectsApp openApp={openApp} />
  case 'projectDetails':
    return <ProjectDetailsApp projectId={win.payload?.projectId} />
  case 'credits':
    return <CreditsApp />
  case 'help':
    return <HelpApp />
  default:
```

6 Boot, BIOS, Failure, and Recovery

The build no longer has only a simple boot screen. Current code models a fuller retro machine: cold boot, BIOS setup, boot-device menu, Windows startup menu, normal boot, Safe Mode, DOS-only mode, recovery, crash, and shutdown. App.tsx renders those screens, but the reducer decides where the boot lands.

- 1 createDefaultState starts at phase 'boot' with bootTarget 'normal' and BIOS settings from persistence or defaults.
- 2 BootScreen runs the visual startup sequence and calls finishBoot.
- 3 The reducer checks bootTarget and filesystem health, then routes to desktop, bootMenu, dosOnly, recovery, or crash.

-
- 4 Deleting a critical Windows file changes the VFS first, then store crashes the OS after a short delay.
-
- 5 Recovery restores missing protected system files from the initial filesystem snapshot and then normal boot can succeed again.

`src/os/store.tsx` *finishBoot routing*

```
case 'FINISH_BOOT': {
  if (state.phase === 'boot') return state
  switch (state.bootTarget) {
    case 'normal':
      if (!isSystemHealthy(state.fs)) {
        return { ...state, phase: 'bootMenu', bootTarget: 'normal' }
      }
      return { ...state, phase: 'desktop', bootMode: 'normal' }
    case 'safe':
      if (shouldSafeModeBlueScreen(state.fs)) {
        return {
          ...state,
          phase: 'crashed',
          bootMode: 'safe',
          crash: safeModeCrash(missingRequiredSystemFiles(state.fs)),
        }
      }
      return {
        ...state,
        phase: 'desktop',
        bootMode: 'safe',
        network: releasedNetworkState(),
      }
    case 'dos':
      return { ...state, phase: 'dosOnly' }
    case 'recovery':
      return { ...state, phase: 'recovery' }
  }
  return state
}
```

7 Virtual Filesystem

The virtual filesystem is a pure data engine. It normalizes Windows-style paths, stores every folder/file as an `FsNode` keyed by path, tracks recycle-bin entries, blocks writes inside protected Windows paths, and resolves file associations back to apps.

Filesystem behavior	Why it exists
Normalized path helpers	Apps and commands can accept loose user input but operate on consistent <code>C:\</code> style paths.
Pure operations returning <code>FsResult</code>	Apps can show errors without catching exceptions, and tests can verify logic without <code>React</code> .
Soft delete into recycle entries	Explorer/Recycling behavior feels real and allows restore.
Critical-file detection	Deleting kernel/system files intentionally triggers crash/recovery gameplay.
<code>openTargetFor</code> associations	Double-clicking folders/files routes through the same app-opening pipeline as desktop icons.
<code>C:\Windows\Desktop</code> mirror	Filesystem-created shortcuts become visible desktop icons.

`src/os/filesystem.ts` *protected roots, desktop folder, and associations*

```
export const PROTECTED_ROOT = 'C:\\Windows'

/** Files that Windows needs to boot. Deleting any of them is... educational. */
export const REQUIRED_SYSTEM_FILES: string[] = [
  'C:\\Windows\\System32\\kernel32.dll',
  'C:\\Windows\\System32\\user32.dll',
  'C:\\Windows\\System32\\gdi32.dll',
  'C:\\Windows\\System32\\shell32.dll',
  'C:\\Windows\\System32\\vmm32.vxd',
  'C:\\Windows\\System32\\display.drv',
  'C:\\Windows\\EXPLORER.EXE',
  'C:\\Windows\\Command\\COMMAND.COM',
  'C:\\Windows\\WIN.INI',
  'C:\\Windows\\SYSTEM.INI',
]

const ACCESS_DENIED = 'Access is denied. The file is being used by Windows.'

// -----
// Path utilities
// -----

export const DESKTOP_FOLDER = 'C:\\Windows\\Desktop'

/**
 * Drops a `.lnk` shortcut for `targetPath` into the Desktop folder so it shows up
 * as a desktop icon. Uses internalInsertNode to bypass the C:\\Windows write
 * protection (a "Send to Desktop" is a deliberate, safe system op, like restore).
 */
export function createDesktopShortcut(fs: FsState, targetPath: string): FsResult {
  const target = getNode(fs, targetPath)
  if (!target) {
    return fail(fs, 'The system cannot find the file specified.')
  }
}
```

```
const desktop = getNode(fs, DESKTOP_FOLDER)
if (!desktop || desktop.kind !== 'folder') {
  return fail(fs, 'The system cannot find the path specified.')
}

const open = openTargetFor(target)
const baseLabel = target.name.replace(/\.lnk$/i, '')
const name = uniqueChildName(fs, desktop.path, `${baseLabel}.lnk`)
const path = joinPath(desktop.path, name)
const node: FsNode = {
  path,
  name,
  kind: 'file',
  icon: target.icon,
  fileType: 'Shortcut',
  size: 1024,
  modified: nowStamp(),
  appId: open?.appId ?? target.appId,
  appPayload: open?.payload ?? target.appPayload,
}

return { fs: internalInsertNode(fs, node), error: null, createdPath: path }
}

let recycleCounter = 0

export function deleteNode(fs: FsState, path: string): { fs: FsState; error: string | null; }

export function openTargetFor(node: FsNode): { appId: AppId; payload: WindowPayload } | null {
  if (node.appId) {
    return { appId: node.appId, payload: node.appPayload ?? {} }
  }

  if (node.kind === 'folder') {
    return { appId: 'explorer', payload: { path: node.path } }
  }

  const ext = extensionOf(node.name)
  if (TEXT_EXTENSIONS.has(ext)) {
    return { appId: 'notepad', payload: { filePath: node.path } }
  }
}
```

```
}  
if (DOCUMENT_EXTENSIONS.has(ext)) {  
  return { appId: 'wordpad', payload: { filePath: node.path } }  
}  
if (IMAGE_EXTENSIONS.has(ext)) {  
  return { appId: 'imageView', payload: { filePath: node.path } }  
}  
if (AUDIO_EXTENSIONS.has(ext)) {  
  return { appId: 'mediaPlayer', payload: { filePath: node.path } }  
}  
if (VIDEO_EXTENSIONS.has(ext)) {  
  return { appId: 'videoPlayer', payload: { filePath: node.path } }  
}  
if (WEB_EXTENSIONS.has(ext)) {  
  return { appId: 'internetExplorer', payload: { url: node.content ?? 'about:home' } }  
}
```

8 CSS and UI Build Strategy

The CSS refactor removed the old monolithic App.css. Global shell styling now loads eagerly in main.tsx in a deliberate cascade order. Each app imports its own stylesheet at the top of its component file, so Vite places that CSS with the lazy app chunk. This keeps the initial desktop shell available without pulling every app's styling into the first interactive moment.

CSS layer	Files
Third-party base	98.css/dist/98.css
Global foundation	src/index.css, src/styles/base.css, cursors.css, common.css, responsive.css
Shell/system eager CSS	BootScreen.css, CrashScreen.css, WindowFrame.css, StartMenu.css, Taskbar.css, desktop.css, bios.css, message-box.css
Shared app surface CSS	src/styles/file-manager.css and other shared shell utility sheets
Lazy app CSS	src/components/apps/*App.css imported by each app component

Build verification showed this split in production output: the Vite build emitted a main index CSS file plus separate CSS assets such as CalculatorApp, ExplorerApp, PaintApp, InternetExplorerApp, WordPadApp, and other app chunks.

9 Integrations and Browser APIs

The app keeps dependencies intentionally small. 98.css supplies authentic Windows 98 controls. js-dos supplies the WASM DOSBox runtime for DOOM and Wolfenstein 3D. Everything else uses the browser platform directly.

Integration	Current role in the app
98.css	Base Win98 widgets: windows, title bars, buttons, panels, status bars, menus, inputs.
js-dos	Self-hosted emulator runtime and .jsdos game bundles; lazy-loaded by JsDosGameApp.
Web Audio / OfflineAudioContext	Synthesized system sounds and WAV rendering without Microsoft audio samples.
MediaRecorder / getUserMedia	Real microphone capture in Sound Recorder.
Canvas 2D + DataView	Paint drawing, BMP export, and Task Manager charts.
Wayback iframe + fetch	Internet Explorer archive browsing plus live Wikipedia-backed search results.
localStorage	Persisted OS snapshot under win98-portfolio.v4, including filesystem, BIOS, theme, wallpaper, cursor, audio, network, and icon positions.

10 How the Build Was Verified

For this phase, the documentation was checked against source files rather than only against Markdown prose. The project also passed the normal validation commands in the current worktree before this rewrite phase began.

Check	Result
Markdown/source audit	Build-doc sources read; current code inspected for phase, app, CSS, persistence, filesystem, and integration facts.
App registry audit	25 app definitions found in src/data/apps.ts.
Lint	npm.cmd run lint completed successfully in the current worktree.
Unit tests	npm.cmd test completed successfully: 1 file, 22 tests passed.
Production build	npm.cmd run build completed successfully and emitted lazy app JS/CSS assets.
Browser smoke test	Vite desktop booted, taskbar rendered, My Computer opened, and console errors were empty.
DOCX visual render	Skipped unless LibreOffice/soffice is installed; structural DOCX and accessibility checks are still run.

11 Source Evidence Used

The Markdown files were treated as background and intent; the code files below were used as the authority for current behavior. This section is intentionally last so the document reads as an explanation first and an evidence map second.

Source	What it verified
docs/README.md, docs/overview.md	Project purpose, reading path, high-level architecture, and key feature claims.
docs/how-it-was-built.md	Contract-first build narrative, git-phase framing, and recurring patterns.
docs/plugins-and-integrations.md	Dependency choices and browser API integrations.
docs/writing-style-and-conventions.md	Facade hook, pure logic/effect separation, app contract, comment style.
docs/OS_CONTRACT.md	Original binding contract and design intent, checked against current source before use.
src/main.tsx, src/App.tsx	Entry point, eager CSS order, phase router, desktop shell, lazy app loading.
src/types.ts, src/os/useOs.ts, src/os/store.tsx	Current state model, facade API, reducer, boot flow, window lifecycle, persistence side effects.
src/os/filesystem.ts, src/data/initialFilesystem.ts, src/os/recovery.ts	VFS operations, seed C: drive, critical-file crash and restore loop.
src/data/apps.ts, src/data/themes.ts, src/data/icons.ts	App registry, desktop/start menu data, theme and icon data.
src/components/apps/*, public/js-dos, public/games	Feature implementation details and self-hosted DOS game assets.

Design preset: compact_reference_guide. Page geometry: US Letter, 1 inch margins, Calibri 11 pt body, fixed-width DXA tables with repeating header rows.